Final Deliverable Report
SYSC3303 - A2G07

Jordan Grewal - 101287828
Nolan Kisser - 101222376
Celina Yang - 101299938

April 8, 2026

Table of Contents

Table of Contents **2**

1. Responsibilities / Contributions **3**

1.1 Iteration 1 3

1.2 Iteration 2 3

1.3 Iteration 3 4

1.4 Iteration 4 4

1.5 Iteration 5 4

2. Diagrams **5**

2.1 UML Class Diagram 5

2.2 Scheduler State Machine Diagram 7

2.3 DroneSubsystem State Machine Diagram 8

2.4 Sequence Diagrams Showing All Error Scenarios 9

2.5 Timing Diagrams 10

3. Detailed Setup and Test Instructions **11**

3.1 Setup Option 1: IntelliJ IDEA 11

3.2 Setup Option 2: Command Line 11

3.3 Running Tests Option 1: Run All Tests 11

3.4 Running Tests Option 2: Run Individual Test Classes 11

4. Results from Measurements **12**

4.1 Result Outputs 12

4.2 Analysis 14

5. Conclusion **15**

5.1 Reflection 15

5.2 Video Demo Link 15

1. Responsibilities / Contributions

1.1 Iteration 1

Team Member	Contributions
Jordan	● Responsible for designing and implementing the GUI ● Illustrated the different fire zones where the drones will service events ● Implemented a legend to display different events and drone simulation
Nolan	● Responsible for creating the system diagrams, including the UML and sequence diagram to display the design and structure of the system ● Illustrated the communication between the different subsystems and the system behaviour ● Created Readme file outlining project structure and usage
Celina	● Implemented system code for the simulation with CSV event and zone files ● Integrated zone coordinate processing and calculations for drone times (enroute, extinguishing, return) ● Developed synchronization and communication between the subsystems
Ozan	● Developed and executed unit tests using JUnit to validate system functionality ● Helped in identifying and debugging issues displayed through testing ● Implemented edge cases relating to the subsystems communication

1.2 Iteration 2

Team Member	Contributions
Jordan	● Updated UML class diagram to account for newly implemented logic ● Responsible for designing the state machine diagram to display the Scheduler and DroneSubsystem ● Updated the GUI to display drone states and number of active fires ● Updated logic to handle different faults
Nolan	● Responsible for the main scheduling logic which includes determining which drone handles fires ● Implemented drone state transitions ● Integrated drone status updates and notifications with the Schedule ● Updated unit tests to fix a timeout error to account for the time in the new states
Celina	● Developed and executed unit tests using JUnit to validate system functionality for the updated DroneSubsystem and Scheduling logic ● Implemented getter functions in DroneSubsystem for unit tests ● Updated README file for iteration 2 ● Update sequence diagram to implement iteration 2 logic

1.3 Iteration 3

Team Member	Contributions
Jordan	● Implemented UDP to split the system into separate programs ● Updated program to ensure that communication is through UDP ● Updated GUI
Nolan	● Revised state machine in scheduler based on the feedback received for iteration 2 ● Handled test cases for UDP implementation
Celina	● Implemented UDP to split the system into separate programs ● Updated program to ensure that communication is through UDP ● Revised test cases based on iteration 2 feedback

1.4 Iteration 4

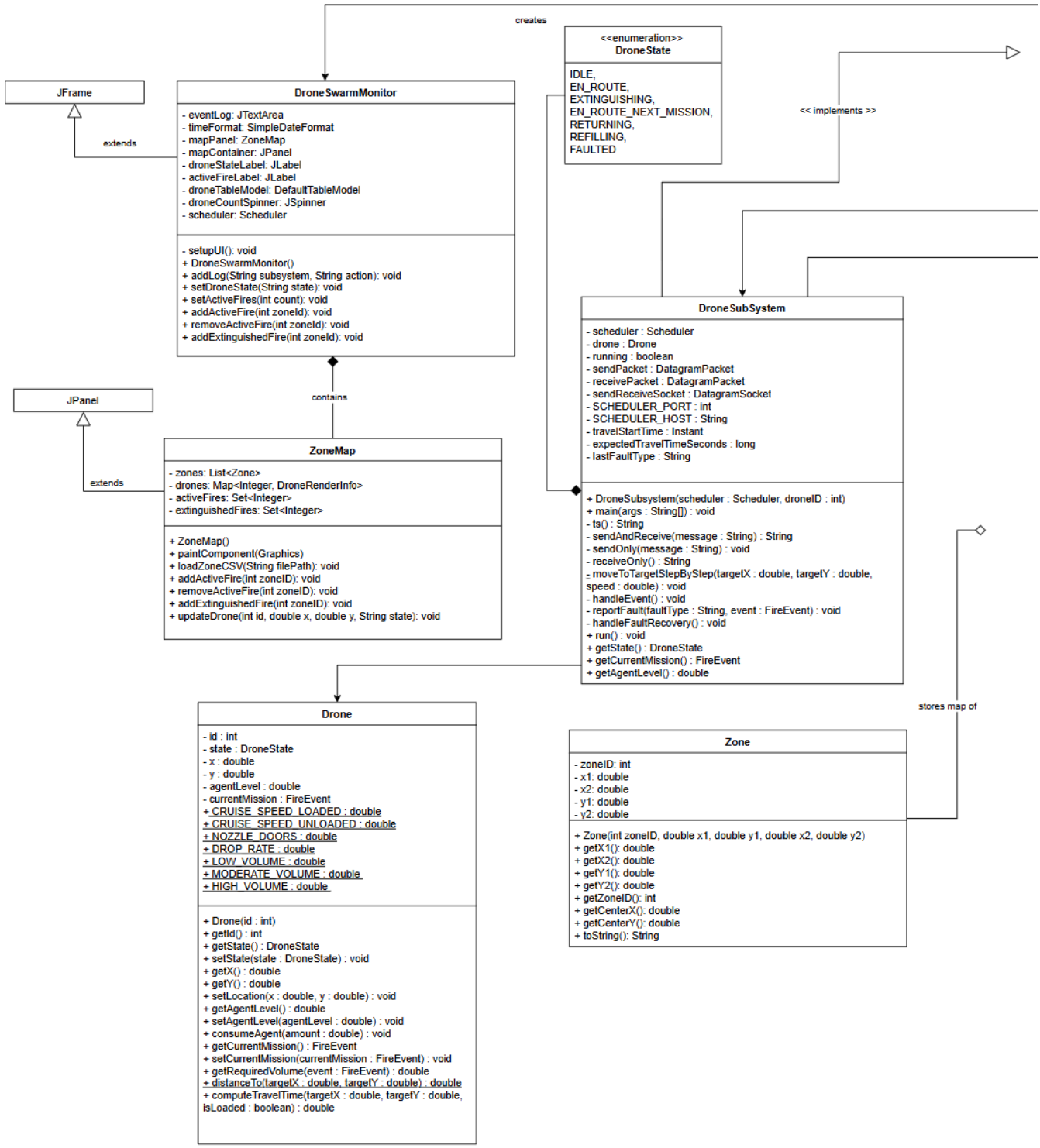
Team Member	Contributions
Jordan	● Developed and executed unit tests using JUnit to validate system functionality regarding updated system logic including fault handling ● Updated state machine to implement newly updated logic
Nolan	● Implemented fault handling logic within the system (nozzle jam, communication loss, and flight failure) ● Integrated fault states into Scheduler and DroneSubsystem
Celina	● Refactored system to implement separate Drone class its own state machine ● Updated GUI to reflect new drone states (ie. OFFLINE) and fault conditions ● Implement timing diagrams to show error scenarios

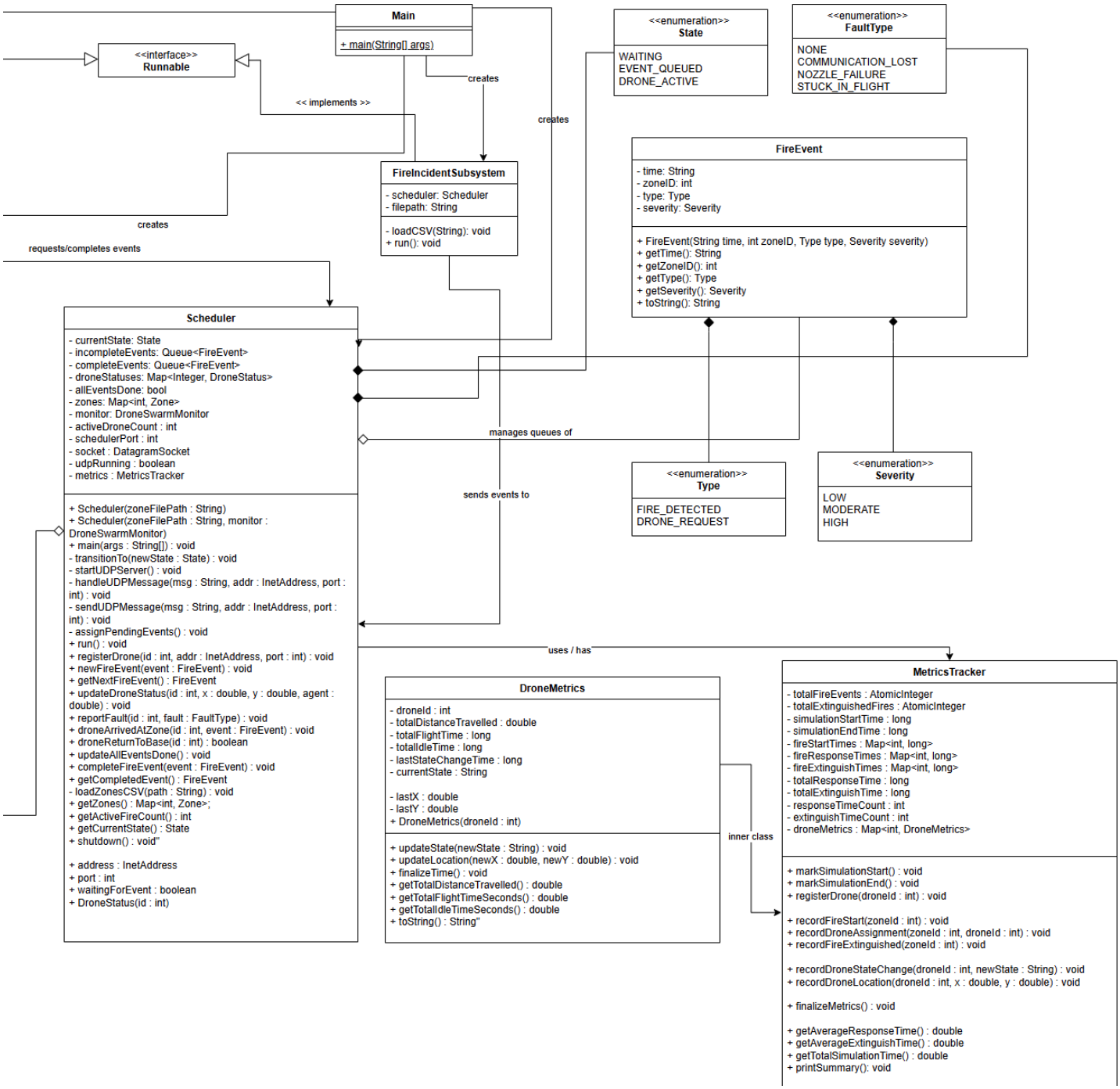
1.5 Iteration 5

Team Member	Contributions
Jordan	● Updated UML class diagrams ● Implemented new state to deal with excess agent leftover
Nolan	● Finalized GUI to include all 5 zones ● Implemented tests and remaining diagrams
Celina	● Implemented performance metrics tracking for drones

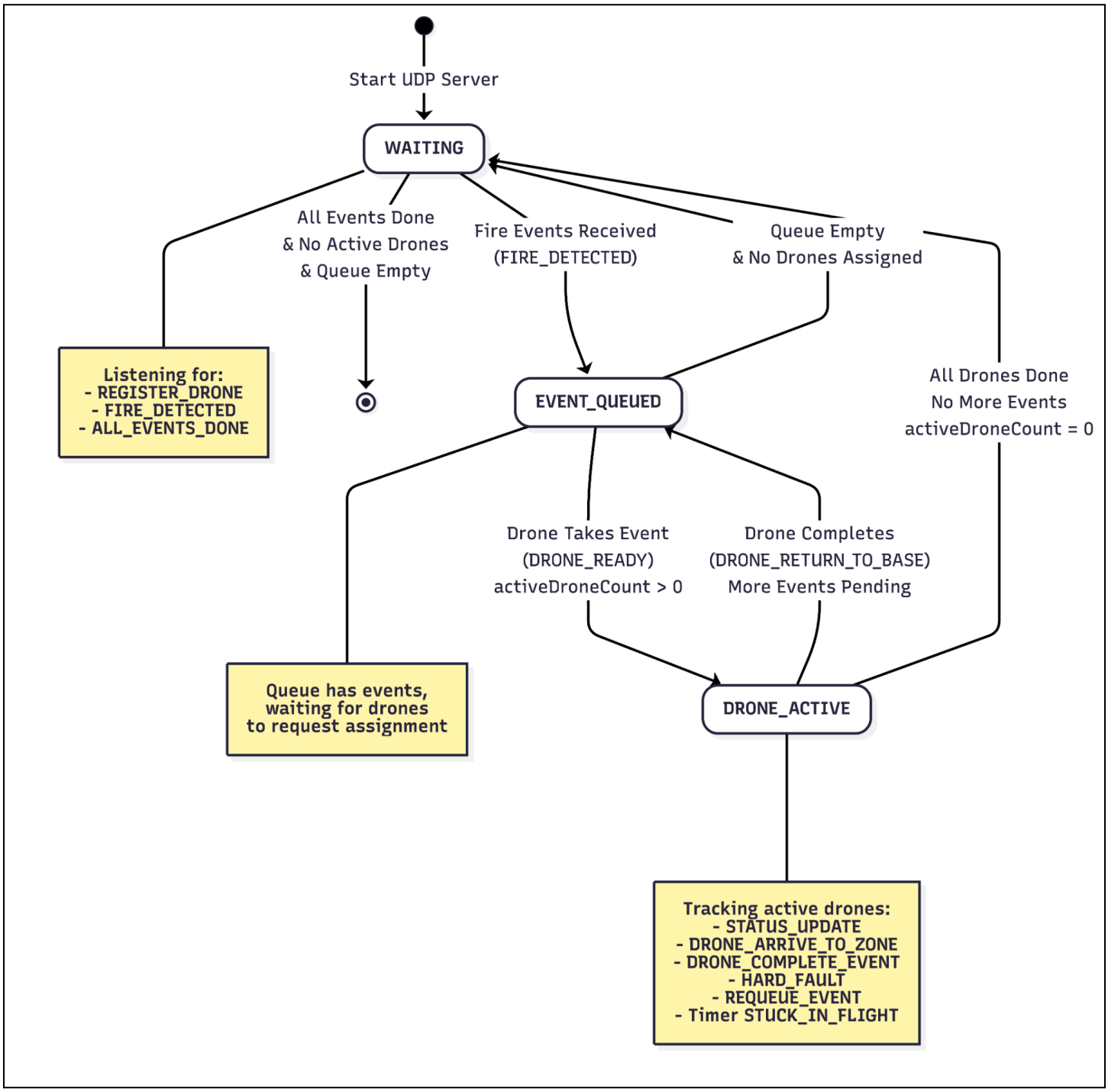
2. Diagrams

2.1 UML Class Diagram

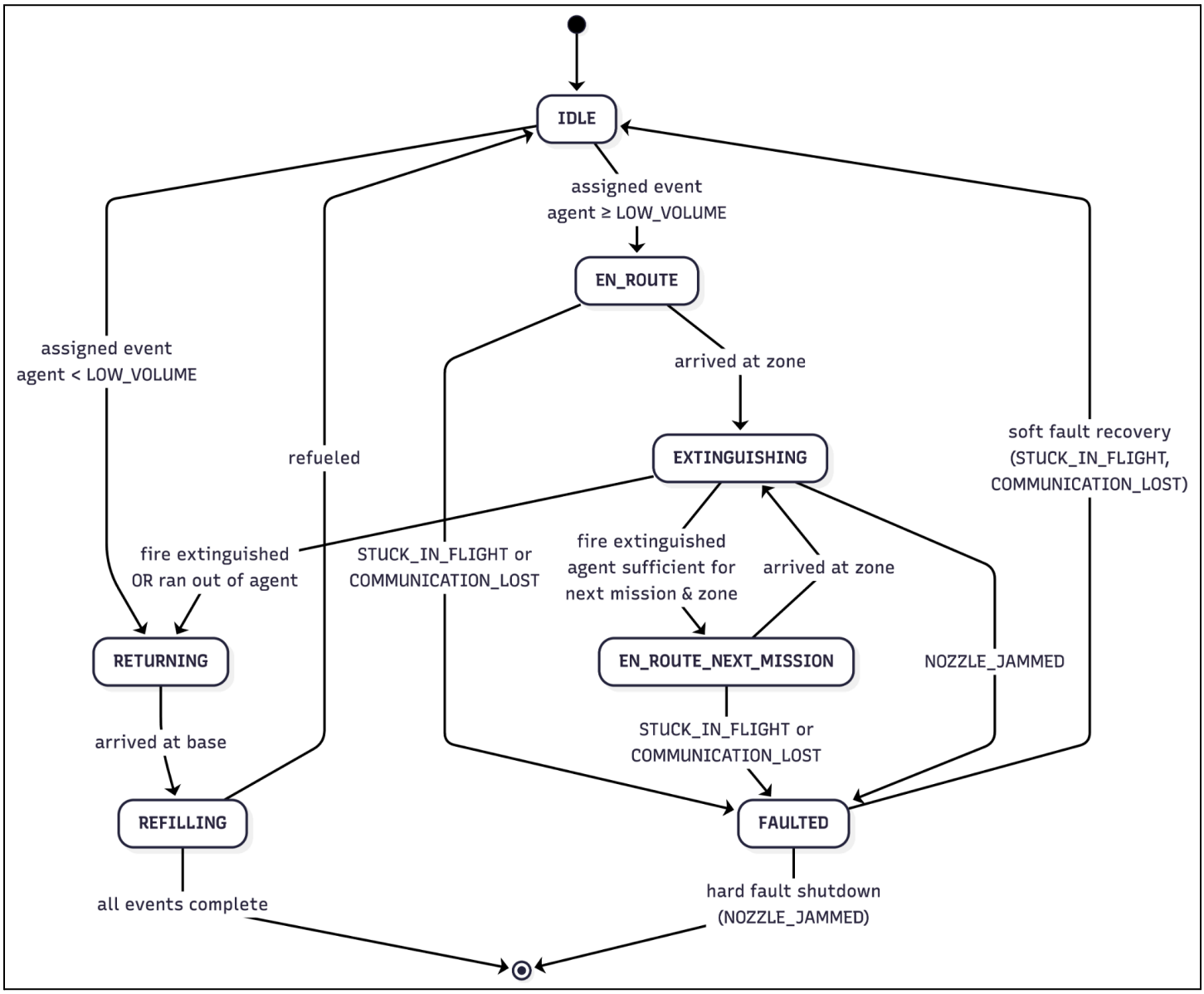



Main
+ main(String[] args)
<<interface>>
Runnable
<< implements >>
creates
creates
creates
FireIncidentSubsystem
- scheduler: Scheduler
- filepath: String
- loadCSV(String): void
+ run(): void
<<enumeration>>
State
WAITING
EVENT_QUEUED
DRONE_ACTIVE
<<enumeration>>
FaultType
NONE
COMMUNICATION_LOST
NOZZLE_FAILURE
STUCK_IN_FLIGHT
FireEvent
- time: String
- zoneID: int
- type: Type
- severity: Severity
+ FireEvent(String time, int zoneID, Type type, Severity severity)
+ getTime(): String
+ getZoneID(): int
+ getType(): Type
+ getSeverity(): Severity
+ toString(): String
requests/completes events
Scheduler
- currentState: State
- incompleteEvents: Queue<FireEvent>
- completeEvents: Queue<FireEvent>
- droneStatuses: Map<Integer, DroneStatus>
- allEventsDone: bool
- zones: Map<int, Zone>
- monitor: DroneSwarmMonitor
- activeDroneCount : int
- schedulerPort : int
- socket : DatagramSocket
- udpRunning : boolean
- metrics : MetricsTracker
+ Scheduler(zoneFilePath : String)
+ Scheduler(zoneFilePath : String, monitor : DroneSwarmMonitor)
+ main(args : String[]) : void
- transitionTo(newState : State) : void
- startUDPServer() : void
- handleUDPMessage(msg : String, addr : InetAddress, port : int) : void
- sendUDPMessage(msg : String, addr : InetAddress, port : int) : void
- assignPendingEvents() : void
+ run() : void
+ registerDrone(id : int, addr : InetAddress, port : int) : void
+ newFireEvent(event : FireEvent) : void
+ getNextFireEvent() : FireEvent
+ updateDroneStatus(id : int, x : double, y : double, agent : double) : void
+ reportFault(id : int, fault : FaultType) : void
+ droneArrivedAtZone(id : int, event : FireEvent) : void
+ droneReturnToBase(id : int) : boolean
+ updateAllEventsDone() : void
+ completeFireEvent(event : FireEvent) : void
+ getCompletedEvent() : FireEvent
- loadZonesCSV(path : String) : void
+ getZones() : Map<int, Zone>;
+ getActiveFireCount() : int
+ getCurrentState() : State
+ shutdown() : void"
+ address : InetAddress
+ port : int
+ waitingForEvent : boolean
+ DroneStatus(id : int)
manages queues of
sends events to
<<enumeration>>
Type
FIRE_DETECTED
DRONE_REQUEST
<<enumeration>>
Severity
LOW
MODERATE
HIGH
uses / has
DroneMetrics
- droneId : int
- totalDistanceTravelled : double
- totalFlightTime : long
- totalIdleTime : long
- lastStateChangeTime : long
- currentState : String
- lastX : double
- lastY : double
+ DroneMetrics(droneId : int)
+ updateState(newState : String) : void
+ updateLocation(newX : double, newY : double) : void
+ finalizeTime() : void
+ getTotalDistanceTravelled() : double
+ getTotalFlightTimeSeconds() : double
+ getTotalIdleTimeSeconds() : double
+ toString() : String"
inner class
MetricsTracker
- totalFireEvents : AtomicInteger
- totalExtinguishedFires : AtomicInteger
- simulationStartTime : long
- simulationEndTime : long
- fireStartTimes : Map<int, long>
- fireResponseTimes : Map<int, long>
- fireExtinguishTimes : Map<int, long>
- totalResponseTime : long
- totalExtinguishTime : long
- responseTimeCount : int
- extinguishTimeCount : int
- droneMetrics : Map<int, DroneMetrics>
+ markSimulationStart() : void
+ markSimulationEnd() : void
+ registerDrone(droneId : int) : void
+ recordFireStart(zoneId : int) : void
+ recordDroneAssignment(zoneId : int, droneId : int) : void
+ recordFireExtinguished(zoneId : int) : void
+ recordDroneStateChange(droneId : int, newState : String) : void
+ recordDroneLocation(droneId : int, x : double, y : double) : void
+ finalizeMetrics() : void
+ getAverageResponseTime() : double
+ getAverageExtinguishTime() : double
+ getTotalSimulationTime() : double
+ printSummary(): void

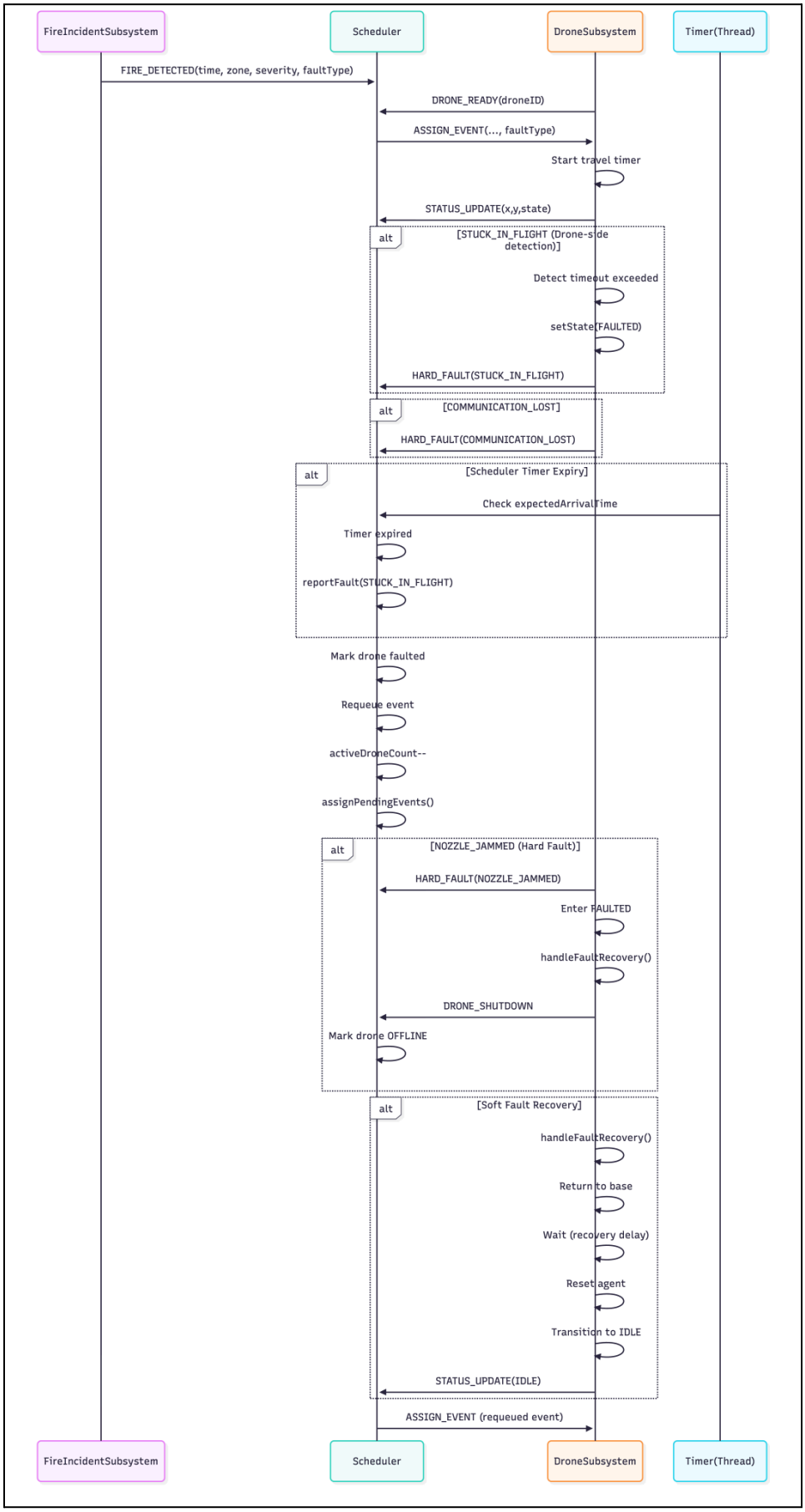
2.2 Scheduler State Machine Diagram



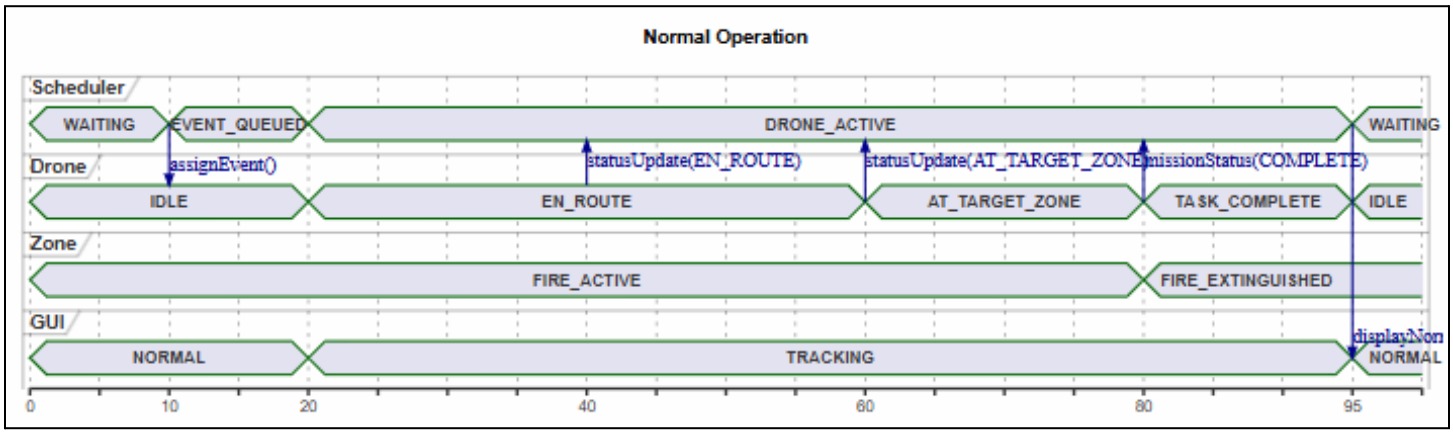
2.3 DroneSubsystem State Machine Diagram

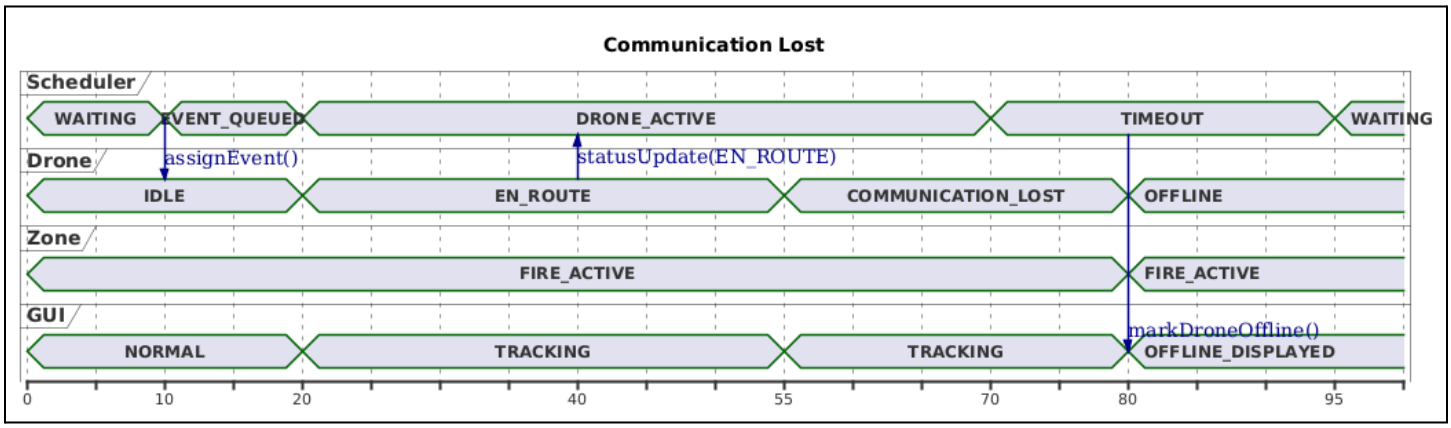


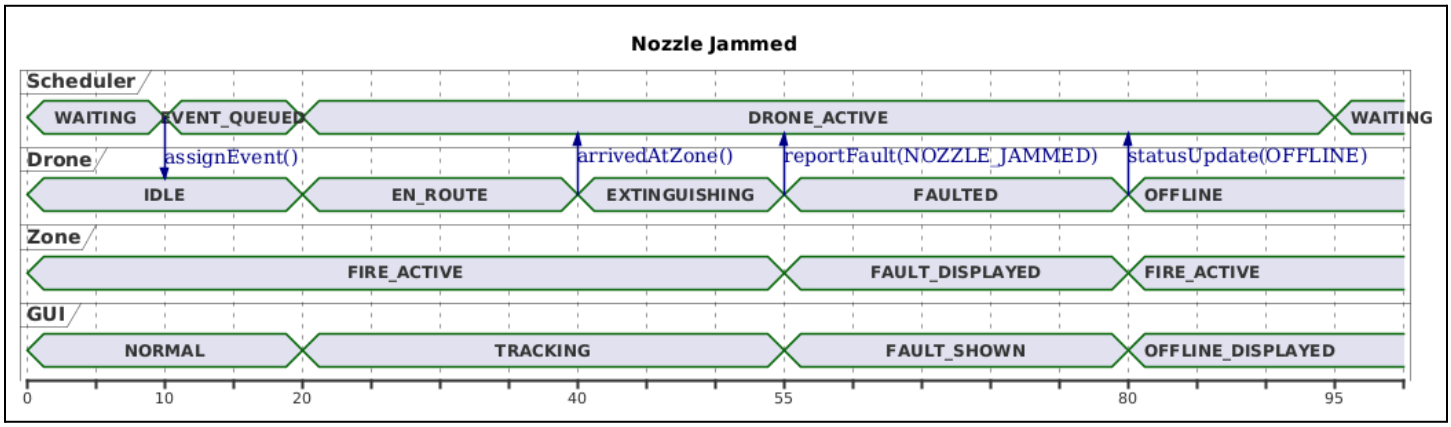
2.4 Sequence Diagrams Showing All Error Scenarios

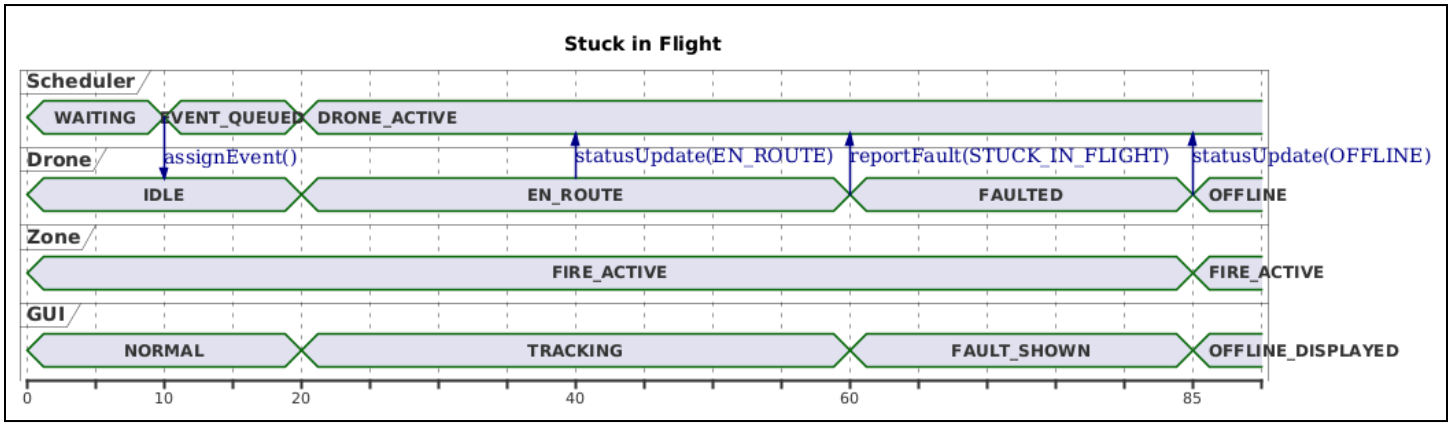


2.5 Timing Diagrams









3. Detailed Setup and Test Instructions

3.1 Setup Option 1: IntelliJ IDEA

1. Open IntelliJ IDEA
2. Select File > Open and navigate to the `FireFightingDroneSwarm` folder
3. Ensure the Project SDK is set to Java 17+ (File > Project Structure > Project > SDK)
4. If using Maven, the project will automatically download dependencies. Otherwise, add JUnit 5.10.1 manually:
    - Go to File > Project Structure > Libraries
    - Click + and select From Maven…
    - Search for `org.junit.jupiter:junit-jupiter:5.10.1` and add it
5. Right-click and run `Scheduler.java`
6. Once GUI appears, input desired number of drones in "Number of Drones"
7. Click "`Launch Simulation`" to start the simulation

3.2 Setup Option 2: Command Line

1. Navigate to the src directory:
    - `cd src`
2. Compile the Java files:
    - `javac model/*.java subsystems/*.java ui/*.java Main.java`
3. Run the application by starting each subsystem in a separate terminal:
    - Terminal 1 (Scheduler):
        - `java subsystems.Scheduler`
4. Once GUI appears, input desired number of drones in "Number of Drones"
5. Click "`Launch Simulation`" to start the simulation

3.3 Running Tests Option 1: Run All Tests

1. In IntelliJ, right-click on the `test` folder in your Project view
2. Select "Run 'All Tests'"
3. View results in the Run window

3.4 Running Tests Option 2: Run Individual Test Classes

Right-click on any test class and select Run:

- `FireEventTest` - Tests FireEvent data structure
- `ZoneTest` - Tests Zone coordinate calculations
- `SchedulerTest` - Tests scheduler synchronization and event management
- `DroneSubsystemTest` - Tests drone behavior
- `FireIncidentSubsystemTest` - Tests CSV parsing and event submission
- `SystemIntegrationTest` - Tests complete system integration

4. Results from Measurements

4.1 Result Outputs

Runs	Output
1	=== PERFORMANCE METRICS SUMMARY === Total Zones With Fire Events: 5 Total Extinguished Fires: 50 Average Response Time: 0.00 seconds Average Extinguish Time: 22.33 seconds Total Time to Extinguish All Fires: 52.72 seconds --- Individual Drone Metrics --- Drone 1: Distance=35700.00 m, Flight Time=43.10 s, Idle Time=10.51 s Drone 2: Distance=30700.00 m, Flight Time=38.98 s, Idle Time=14.63 s Drone 3: Distance=33900.00 m, Flight Time=40.15 s, Idle Time=13.45 s Drone 4: Distance=38900.00 m, Flight Time=44.58 s, Idle Time=9.03 s Drone 5: Distance=34200.00 m, Flight Time=38.88 s, Idle Time=14.72 s Drone 6: Distance=20000.00 m, Flight Time=26.62 s, Idle Time=26.98 s Drone 7: Distance=19700.00 m, Flight Time=23.29 s, Idle Time=30.31 s Drone 8: Distance=10700.00 m, Flight Time=12.46 s, Idle Time=41.15 s Drone 9: Distance=6400.00 m, Flight Time=7.02 s, Idle Time=46.59 s Drone 10: Distance=3900.00 m, Flight Time=4.92 s, Idle Time=48.69 s
2	=== PERFORMANCE METRICS SUMMARY === Total Zones With Fire Events: 5 Total Extinguished Fires: 50 Average Response Time: 0.00 seconds Average Extinguish Time: 27.27 seconds Total Time to Extinguish All Fires: 58.92 seconds --- Individual Drone Metrics --- Drone 1: Distance=42400.00 m, Flight Time=49.36 s, Idle Time=10.04 s Drone 2: Distance=37900.00 m, Flight Time=46.48 s, Idle Time=12.92 s Drone 3: Distance=38500.00 m, Flight Time=44.29 s, Idle Time=15.12 s Drone 4: Distance=27200.00 m, Flight Time=34.36 s, Idle Time=25.04 s Drone 5: Distance=28900.00 m, Flight Time=35.77 s, Idle Time=23.63 s Drone 6: Distance=25300.00 m, Flight Time=29.77 s, Idle Time=29.64 s Drone 7: Distance=17100.00 m, Flight Time=19.51 s, Idle Time=39.89 s Drone 8: Distance=12500.00 m, Flight Time=15.30 s, Idle Time=44.11 s Drone 9: Distance=4300.00 m, Flight Time=5.31 s, Idle Time=54.09 s Drone 10: Distance=0.00 m, Flight Time=0.00 s, Idle Time=59.40 s
3	=== PERFORMANCE METRICS SUMMARY === Total Zones With Fire Events: 5 Total Extinguished Fires: 50 Average Response Time: 0.00 seconds Average Extinguish Time: 25.89 seconds Total Time to Extinguish All Fires: 56.74 seconds --- Individual Drone Metrics --- Drone 1: Distance=39600.00 m, Flight Time=48.28 s, Idle Time=8.94 s

	Drone 2: Distance=35000.00 m, Flight Time=44.05 s, Idle Time=13.17 s Drone 3: Distance=34300.00 m, Flight Time=40.30 s, Idle Time=16.93 s Drone 4: Distance=31700.00 m, Flight Time=36.75 s, Idle Time=20.48 s Drone 5: Distance=25400.00 m, Flight Time=31.23 s, Idle Time=26.00 s Drone 6: Distance=27400.00 m, Flight Time=31.46 s, Idle Time=25.76 s Drone 7: Distance=26100.00 m, Flight Time=30.32 s, Idle Time=26.90 s Drone 8: Distance=7800.00 m, Flight Time=9.95 s, Idle Time=47.28 s Drone 9: Distance=6800.00 m, Flight Time=7.47 s, Idle Time=49.75 s Drone 10: Distance=0.00 m, Flight Time=0.00 s, Idle Time=57.22 s
4	=== PERFORMANCE METRICS SUMMARY === Total Zones With Fire Events: 5 Total Extinguished Fires: 50 Average Response Time: 0.00 seconds Average Extinguish Time: 24.92 seconds Total Time to Extinguish All Fires: 58.86 seconds --- Individual Drone Metrics --- Drone 1: Distance=40000.00 m, Flight Time=48.46 s, Idle Time=12.31 s Drone 2: Distance=37400.00 m, Flight Time=44.88 s, Idle Time=15.89 s Drone 3: Distance=46500.00 m, Flight Time=52.55 s, Idle Time=8.22 s Drone 4: Distance=32400.00 m, Flight Time=40.43 s, Idle Time=20.33 s Drone 5: Distance=31000.00 m, Flight Time=37.81 s, Idle Time=22.96 s Drone 6: Distance=16800.00 m, Flight Time=20.86 s, Idle Time=39.91 s Drone 7: Distance=15400.00 m, Flight Time=18.07 s, Idle Time=42.69 s Drone 8: Distance=10300.00 m, Flight Time=12.12 s, Idle Time=48.65 s Drone 9: Distance=4300.00 m, Flight Time=5.27 s, Idle Time=55.50 s Drone 10: Distance=0.00 m, Flight Time=0.00 s, Idle Time=60.76 s
5	=== PERFORMANCE METRICS SUMMARY === Total Zones With Fire Events: 5 Total Extinguished Fires: 50 Average Response Time: 0.00 seconds Average Extinguish Time: 22.30 seconds Total Time to Extinguish All Fires: 55.55 seconds --- Individual Drone Metrics --- Drone 1: Distance=37200.00 m, Flight Time=42.90 s, Idle Time=12.87 s Drone 2: Distance=34200.00 m, Flight Time=43.35 s, Idle Time=12.43 s Drone 3: Distance=35000.00 m, Flight Time=39.80 s, Idle Time=15.98 s Drone 4: Distance=29700.00 m, Flight Time=36.43 s, Idle Time=19.34 s Drone 5: Distance=29200.00 m, Flight Time=34.71 s, Idle Time=21.06 s Drone 6: Distance=27100.00 m, Flight Time=32.79 s, Idle Time=22.99 s Drone 7: Distance=21400.00 m, Flight Time=24.60 s, Idle Time=31.17 s Drone 8: Distance=5700.00 m, Flight Time=7.99 s, Idle Time=47.79 s Drone 9: Distance=8200.00 m, Flight Time=10.18 s, Idle Time=45.59 s Drone 10: Distance=6400.00 m, Flight Time=7.13 s, Idle Time=48.64 s
Results	Average Response Time: 0.00s Average Extinguish Time: 24.54s Average Total Time to Extinguish All Fires: 56.56s

4.2 Analysis

The average response time is 0.00 seconds, which indicates that fires are assigned to drones immediately when they become available. With the implementation of the scheduler, the results show that the scheduler is highly responsive and assigns fire events as soon as drones send DRONE_READY messages to indicate that the drone is ready to receive an event and events are available.

The average extinguish time is approximately 24.54 seconds, which indicates the time taken from when a fire is detected to when it is fully extinguished. With the implementation of the drone subsystem and scheduling logic, it shows that drones are able to efficiently travel to fire zones and extinguish fire events within a consistent time range.

The average total time to extinguish all fires is approximately 56.56 seconds, which represents the overall time required for the system to handle all fire events from start to finish. With the current scheduling approach, the results show that the system is able to process multiple events concurrently using multiple drones, resulting in efficient overall completion.

The individual drone metrics show that some drones have higher flight times and distances travelled compared to others, while some drones experience higher idle times. With the implementation of the scheduler, the results show that tasks are assigned to drones based on availability rather than workload balancing, which leads to uneven distribution of work across drones.

5. Conclusion

5.1 Reflection

The system is designed as an asynchronous, distributed style application like the Client-Server pattern. It leverages concurrent threads for the Scheduler, DroneSubSystem, and FireIncidentSubstem and uses UDP (DatagramSocket) for inter-process communication.

Strength of the Design

1. Highly Decoupled and Distributed style architecture, this makes the system highly scalable. A DroneSubsystem could run on entirely different physical machines from Scheduler, perfectly simulating real-world drone swarm application.
2. Robust Event-driven State machines make concurrent logic predictable thus avoiding race conditions and irregular drone behavior.
3. Reliable software design practice by injecting faults and dedicated metrics tracker, this makes debugging, testing and evaluation analysis reliable.

Areas of Improvement

1. String-Based communication in which messages are sent as comma-separated strings and are manually separated. The drawback of this approach is if a message contains an unexpected comma or a missing field, it will throw an exception. This can be improved by using formal serialization such as JSON.
2. UDP error handling, since UDP has no guaranteed delivery, simply dropping a corrupted message can leave a drone waiting forever. We could implement an acknowledgement protocol with a timeout and retry.
3. Hardcoded environment variables into the classes such as “localhost”, “fileName”, etc. We should extract these constants and move into a centralized config.jave class.

5.2 Video Demo Link

https://1drv.ms/v/c/6fa49ed1d3b0bc00/IQBGRzNc8abyRLdc-Lgno7sFAXaI7-Z3ZZZAlaSvfxG00LM?e=ZJ0z3w